Advanced Python for Data Science

Week(6): Cloud Deployment & DevOps

Thursday, 23rd July 2026, 9:30-12:45

Dr. Ghadeer Mobasher

ROADMAP

What We'll Cover Today

1. **Cloud Service Models**
   IaaS vs PaaS vs SaaS, who manages what

2. **CI/CD Pipelines**
   Automated build, test & release on every push

3. **GitHub Actions / GitLab CI**
   Writing the automation recipes → Workflows, jobs, runners & steps. The same idea, different vocabulary

4. **Container Orchestration**
   Managing containers at once

5. **Kubernetes Basics**
   Pods, Deployments, Services & self-healing

Builds on Week 5: we take the Dockerized Flask API and put it behind a pipeline, then deploy it onto Kubernetes.

Renting an Apartment vs. Booking a Hotel Room

Cloud service models differ in how much you manage yourself vs. how much the provider manages for you. Exactly like the difference between an empty apartment, a furnished apartment, and a hotel room.

IaaS

Empty Apartment

You get the walls & plumbing (servers, storage, networking). You bring your own furniture (OS, runtime, app).

e.g. AWS EC2, Google Compute Engine

PaaS

Furnished Apartment

Move-in ready. The landlord handles plumbing & furniture (OS, runtime); you bring your own life (code).

e.g. Google App Engine, AWS Elastic Beanstalk

SaaS

Hotel Room

Fully serviced. You just show up and use it, everything else is the provider's job.

e.g. Gmail, Slack, Notion

A managed Kubernetes service sits in between IaaS and PaaS, the provider runs the control plane, you define what runs inside it.

CI/CD Pipelines

CI **Continuous Integration**

Every push is automatically built and tested; bugs are caught before they reach other developers.

CD **Continuous Delivery**

Every change that passes CI is packaged into a release that could be deployed; a human still clicks "deploy."

CD **Continuous Deployment**

Every change that passes CI is deployed to production automatically; no human step in between.

A TYPICAL PIPELINE

Why it matters for data science?

Model-serving code changes as often as any other software → CI/CD stops a broken preprocessing step from silently reaching production, and (with Docker) guarantees the exact image that passed tests is the one that ships.

GitHub Actions

- **Workflow** Automated process defined in .github/workflows/*.yml
- **Job** Runs on a fresh VM called a runner
- **Step** One command inside a job, executed in order
- **Trigger** on: push, on: pull_request, or a schedule

Runs test_app.py against app.py with pytest.

1. git init in the Week6 directory
2. Create an empty GitHub repo, add remote
3. git add . && git commit -m "..."
4. git push -u origin main
5. Check the Actions tab for the green check ✅

.github/workflows/ci.yml

```
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5
      - uses: actions/setup-python@v6
        with: { python-version: "3.10" }
      - run: pip install -r requirements.txt
      - run: pytest -v
```

TEST LOCALLY SAME COMMANDS THE RUNNER USES

```
# Run the same checks locally before pushing:
pip install -r requirements.txt
pytest -v
```

Question: Why install dependencies with pip install -r requirements.txt on every single run, instead of reusing a previous environment? What happens to the green check if you break a test?

Beyond a Single Container

One container by hand (docker run) is fine for one thing. Real systems need many, coordinated:

- ✓ **Scheduling** → Deciding which machine runs which container
- ✓ **Scaling** → Adding / removing instances based on load
- ✓ **Self-healing** → Restarting or replacing crashed containers
- ✓ **Service discovery** → Containers find each other; traffic is load-balanced
- ✓ **Rolling updates** → Deploying a new version with zero downtime

- **Docker Compose** is a lightweight, single-machine step toward orchestration → many containers, one YAML file.
- **Kubernetes (next)** is the industry-standard orchestrator across a whole cluster.

docker-compose.yml

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      - APP_VERSION=1.0.0
    restart: unless-stopped

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
```

TRY IT

```
docker compose up --build    # start both services
docker ps                    # see 2 containers
docker compose down          # stop everything
```

Kubernetes: Key Concepts

Term	What it means
Cluster	A set of machines (nodes) run as one unit
Node	A single physical or virtual machine in the cluster
Pod	Smallest deployable unit → usually one container
Deployment	Keeps the desired number of Pods running; handles rolling updates
ReplicaSet	What a Deployment uses internally to keep replica count correct
Service	Stable network endpoint that load-balances across Pods
Namespace	Logical partition of a cluster (dev / staging / prod)

- **Why Pods & Deployments, not just containers?** A Deployment describes desired state ("2 healthy Pods, always").
- Kubernetes' control loop compares desired vs. actual state and fixes drift automatically → self-healing and scaling, across an entire cluster.

Deploying to Kubernetes: 3 Steps

1 Build the image

Point Docker at minikube's own environment, then build the same image from Week 5's Dockerfile.

2 Apply the manifests

deployment.yaml describes 2 replicas; service.yaml exposes them on a stable NodePort.

3 Verify & reach it

Confirm Pods/Deployments/Services are up, then open the URL minikube prints.

TERMINAL

```
eval $(minikube docker-env)                  # build INTO minikube's Docker (Mac/Linux)
docker build -t flask-week6:latest .

kubectl apply -f k8s/deployment.yaml         # 2 replicas of the Flask container
kubectl apply -f k8s/service.yaml            # stable NodePort in front of them

kubectl get pods deployments services        # expect 2 Pods Running
minikube service flask-service --url         # open the printed URL in a browser
```

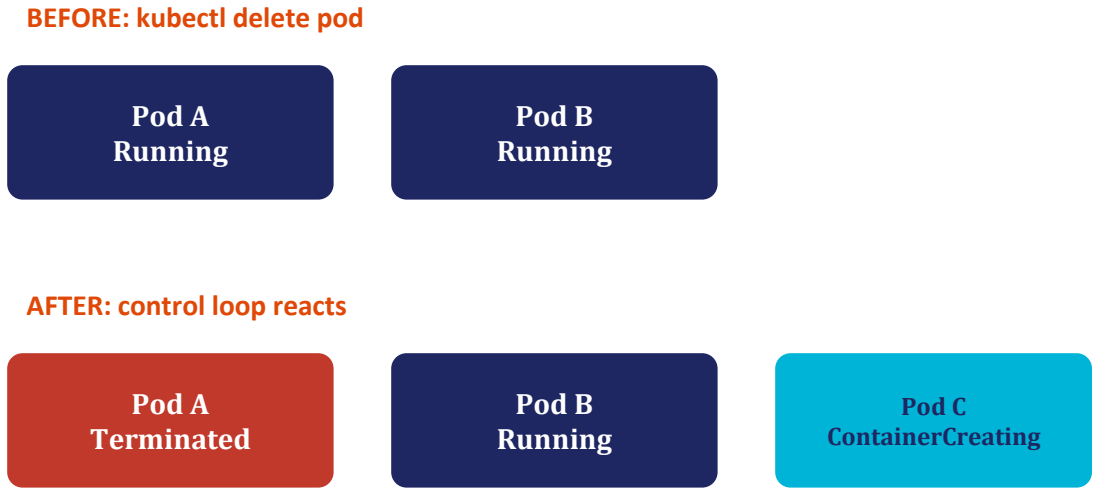
Watch Self-Healing Happen

TERMINAL

```
kubectl get pods
# note one Pod's name

kubectl delete pod <that-pod-name>
# simulate a crash

kubectl get pods
# run again immediately
```



What just happened: the Deployment controller noticed actual state (1 Pod) no longer matched desired state (replicas: 2) and scheduled a replacement → automatically, with no human step.

NOW IT'S YOUR TURN

EXERCISES

week6_exercise.ipynb → small, guided edits to files you already opened, plus a mini challenge that ties everything together.

#	File you edit	Concept practiced
1	.github/workflows/ci.yml	Adding a lint stage (flake8) alongside pytest
2	k8s/deployment.yaml	Scaling replicas + wiring in a ConfigMap
3	k8s/service.yaml	Changing Service type: NodePort → ClusterIP

Key Takeaways

1. Cloud models trade control for convenience: IaaS → PaaS → SaaS.
2. CI/CD turns “it works on my machine” into “it's tested on every push.”
3. GitHub Actions and GitLab CI solve the same problem in different syntax.
4. Kubernetes automates what Docker Compose previews on one machine: scheduling, scaling, self-healing.
5. A Deployment is a promise about desired state → Kubernetes keeps that promise continuously.

Check the starter code in week_6.ipynb

You have 45 minutes to do so

Pause / Break

See you in 15 minutes ☺!
Please be on time.

Team C

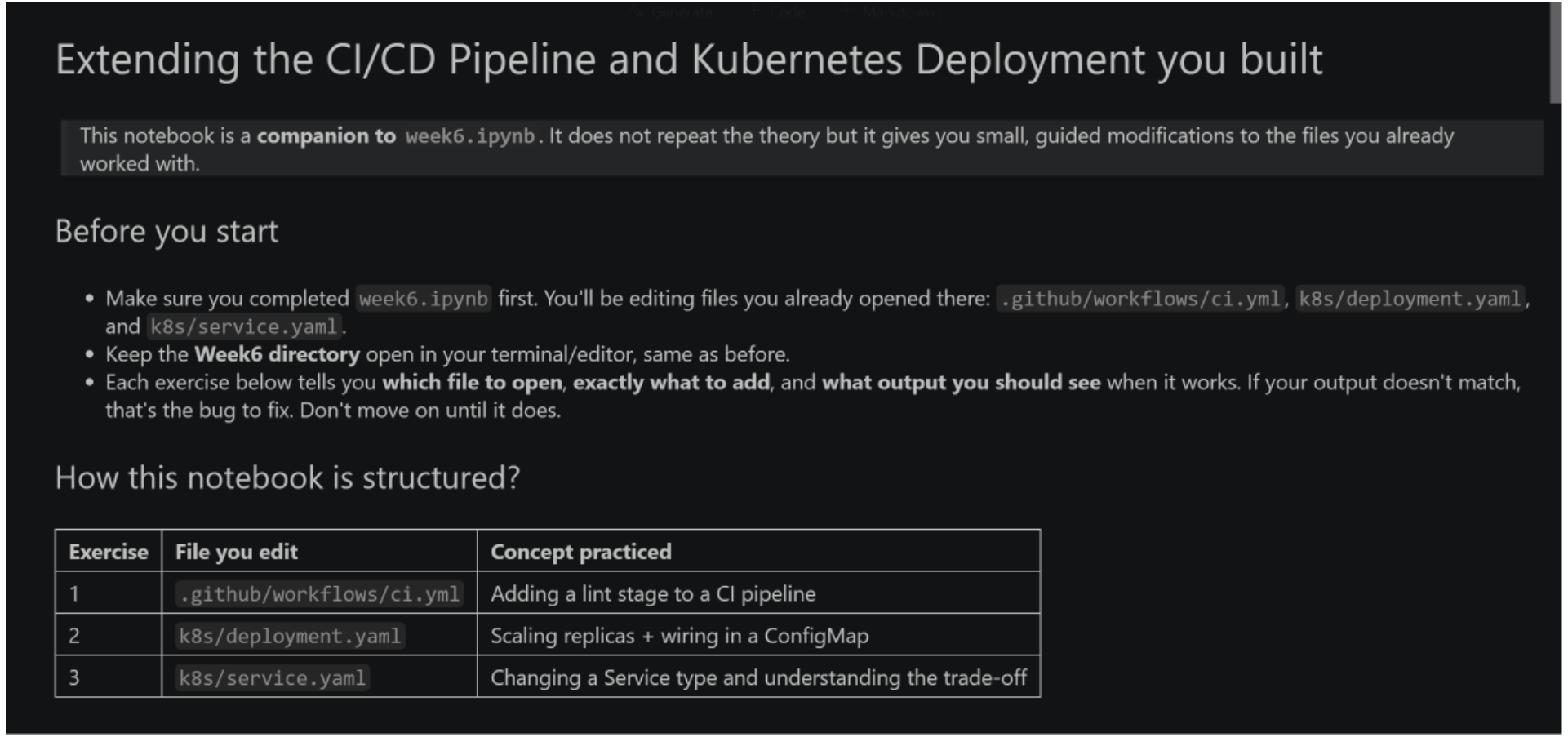
15-20 minutes presentation

Vivek & Gaurav

15-20 minutes presentation

Exercises

Solve Problems (1 -3) in "**week6_Exercise.ipynb**". ***Don't forget to split your screen. Read the whole instructions first and answer the reflection questions in your own words.***



Extending the CI/CD Pipeline and Kubernetes Deployment you built

> This notebook is a **companion to** `week6.ipynb`. It does not repeat the theory but it gives you small, guided modifications to the files you already worked with.

Before you start

- Make sure you completed `week6.ipynb` first. You'll be editing files you already opened there: `.github/workflows/ci.yml`, `k8s/deployment.yaml`, and `k8s/service.yaml`.
- Keep the **Week6 directory** open in your terminal/editor, same as before.
- Each exercise below tells you **which file to open**, **exactly what to add**, and **what output you should see** when it works. If your output doesn't match, that's the bug to fix. Don't move on until it does.

How this notebook is structured?

Exercise	File you edit	Concept practiced
1	`.github/workflows/ci.yml`	Adding a lint stage to a CI pipeline
2	`k8s/deployment.yaml`	Scaling replicas + wiring in a ConfigMap
3	`k8s/service.yaml`	Changing a Service type and understanding the trade-off

Any Questions?

See you next time ☺

Dr. Ghadeer Mobasher
ghadeer.mobasher@srh-hochschulen.de
https://www.linkedin.com/in/ghadeermobasher/
LGS 6, 2. OG, Arc 213